Operating Systems [CSE335]
CPU Scheduler Project

Name	ID
Ali Ahmed Mohamed Abdelmaboud	2200927
Ammar Mohamed Abdelghany	2200707
Ahmed Ramadan Yassin	2200372
Amr Ahmed Wahidi	2200429
Omar Mohamed Hassan	2200294
Hamza Basem Mohamed Ahmed	2200496
Mohamed Elsayed Moustafa	2200095

Table of Contents

Table of Contents ..... **Error! Bookmark not defined.**
1. Introduction 3
2. Objectives 3
3. Project Structure 3
4. Implementation 4
  4.1 Process Model 4
  4.2 Base Scheduler (Abstract Design) 4
  4.3 Engine (Simulation Controller) 4
  4.4 FCFS Scheduling 5
  4.5 Shortest Job First (SJF) 5
  4.6 Priority Scheduling 5
  4.7 Round Robin Scheduling 6
  4.8 Gantt Chart Generation 6
  4.9 GUI Integration 6
5. Scheduling Algorithms — Full Code 8
  5.1 FCFS 8
  5.2 SJF 8
  5.3 Priority 9
  5.4 Round Robin 9
6. Test Cases & Results 11
  6.1 Test Input 11
  6.2 FCFS Trace 11
  6.3 SJF Non-Preemptive Trace 11
  6.4 SJF Preemptive (SRTF) Trace 12
  6.5 Priority Non-Preemptive Trace 12
  6.6 Priority Preemptive Trace 13
  6.7 Round Robin (Quantum = 3) Trace 13
7. Algorithm Comparison 13
8. GUI Design 15
9. How to Run 17
  9.1 From Source 18
  9.2 From the Executable 18
10. Conclusion 18

1. Introduction

This project implements a full CPU Scheduling Simulator with a graphical user interface (GUI) built using Python and Tkinter, the simulator models real CPU scheduling behavior where each unit of simulation time corresponds to one second of real time.

The simulator supports multiple scheduling algorithms including First-Come First-Served (FCFS), Shortest Job First (SJF) in both preemptive and non-preemptive modes, Priority Scheduling in both modes, and Round Robin with a configurable time quantum. It supports dynamic process addition during execution and provides live visual feedback via a Gantt chart and statistics panel.

2. Objectives

- Implement FCFS, SJF (Preemptive & Non-Preemptive), Priority (Preemptive & Non-Preemptive), and Round Robin scheduling algorithms
- Simulate scheduling in real-time with each time unit mapped to one second
- Display a live, updating Gantt chart during execution
- Calculate and display average waiting time and average turnaround time
- Support dynamic process insertion while the scheduler is running
- Provide an instant (non-live) execution mode for rapid results
- Deliver a ready-to-run executable (.exe) desktop GUI application

3. Project Structure

The system follows a modular three-layer architecture separating scheduling logic, simulation execution, and user interface concerns:

```
project/
│
├── core/
│   ├── process.py          # Process data model
│   ├── base_scheduler.py   # Abstract base class for all schedulers
│   ├── engine.py           # Simulation controller
│   ├── fcfs.py             # FCFS algorithm
│   ├── sjf.py              # SJF (preemptive & non-preemptive)
│   ├── priority.py         # Priority scheduling
│   └── roundrobin.py       # Round Robin scheduling
│
├── ui/
│   ├── app.py              # Main application window
│   ├── input_panel.py      # Process input form
│   ├── gantt_chart.py      # Live Gantt chart widget
│   ├── process_table.py    # Live process status table
│   └── stats_panel.py      # Statistics display
│
└── main.py                 # Entry point
```

This separation ensures high maintainability, easy extension when adding new algorithms, and a clear separation of concerns between logic and presentation layers.

4. Implementation

4.1 Process Model

The Process class is the fundamental data structure. It stores all execution-related attributes and provides methods for single-unit execution and time calculation.

```
def execute_one_unit(self):
    self.remaining_time -= 1
    if self.remaining_time == 0:
        self.is_completed = True
        return True
    return False

def calculate_times(self):
    self.turnaround_time = self.completion_time - self.arrival_time
    self.waiting_time    = self.turnaround_time - self.burst_time
```

- execute_one_unit() simulates one time-unit of execution and returns True when the process finishes
- Turnaround Time = Completion Time − Arrival Time
- Waiting Time = Turnaround Time − Burst Time

4.2 Base Scheduler (Abstract Design)

All scheduling algorithms inherit from a common abstract base class that defines a shared interface and provides shared utility methods.

```
class BaseScheduler(ABC):
    def tick(self):
        pass

    def average_waiting_time(self):
        done = [p for p in self.processes if p.is_completed]
        return sum(p.waiting_time for p in done) / len(done)
```

- tick() is overridden by each algorithm to implement its own scheduling logic
- Performance metrics are calculated only over completed processes
- The abstract design ensures consistent interfaces and enables easy addition of new algorithms

4.3 Engine (Simulation Controller)

The Engine acts as a bridge between the GUI and the selected scheduler.

```
def step(self):
    return self.scheduler.tick()   # Advances simulation by 1 second

def run_all(self):
    while not self.scheduler.is_done():
        self.scheduler.tick()       # Instant (non-live) mode
```

- step() executes one simulation tick and is called by the GUI timer every second
- run_all() enables instant execution mode for rapid results without live animation

4.4 FCFS Scheduling

First-Come First-Served is the simplest algorithm. It uses a FIFO queue and executes processes strictly in arrival order. It is non-preemptive.

```
self.current_process = self.ready_queue.pop(0)  # FIFO selection
```

- Non-preemptive: once a process starts, it runs to completion
- Simple and fair in terms of arrival order
- Vulnerable to the convoy effect: short jobs blocked behind long ones

4.5 Shortest Job First (SJF)

SJF selects the process with the shortest remaining burst time. The non-preemptive variant picks the shortest at dispatch time; the preemptive variant (SRTF) re-evaluates every time unit.

```
# Preemptive (SRTF): sort by remaining_time each tick
sorted(available, key=lambda p: (p.remaining_time, p.arrival_time, p.pid))

# Non-Preemptive: sort by original burst_time at dispatch
sorted(available, key=lambda p: (p.burst_time, p.arrival_time, p.pid))
```

- Produces optimal average waiting time among non-preemptive algorithms
- SRTF can preempt a running process when a shorter job arrives
- Potential starvation: long processes may wait indefinitely if short ones keep arriving

4.6 Priority Scheduling

Processes are ordered by a priority value where a lower number means higher importance. Both preemptive and non-preemptive variants are implemented.

```
sorted(processes, key=lambda p: (p.priority, p.arrival_time, p.pid))
```

- Lower priority number = higher importance (as per project requirement)
- Preemptive version: a higher-priority arriving process immediately preempts the current one

- Known limitation: can cause starvation of low-priority processes — no aging mechanism is implemented in the current version

4.7 Round Robin Scheduling

Round Robin assigns each process a fixed time slice (quantum). Once a process exhausts its quantum without finishing, it is returned to the back of the ready queue.

```
if self.time_in_quantum >= self.quantum:
    self.ready_queue.append(self.current_process)  # Re-queue
    self.current_process = None
```

- Default quantum: 5 time units (configurable via GUI input)
- Guarantees no starvation — every process gets CPU time regularly
- A smaller quantum improves responsiveness but increases context-switch overhead; a larger quantum approaches FCFS behaviour
- Context switches between processes arriving at the same boundary are handled before re-queuing the current process

4.8 Gantt Chart Generation

Every scheduler appends one entry to the Gantt chart list per time unit, recording which process (or 'Idle') occupied the CPU.

```
self.gantt_chart.append({
    'pid':  pid,
    'time': self.current_time
})
```

- The chart list is consumed by the GUI to render the live timeline
- Consecutive entries with the same PID are merged visually into a single bar

4.9 GUI Integration

The GUI is implemented with Python's Tkinter library and follows an event-driven model. It interacts with the Engine through a periodic timer.

```
self.start_btn = tk.Button(..., command=self._start_live)

def _tick(self):
    result = self.engine.step()
    self._update_gantt(result)
    self._update_table(result)
    self._update_stats(result)
    if not self.engine.scheduler.is_done():
        self.after(1000, self._tick)   # Schedule next tick in 1 second
```

Key GUI features:

- Scheduler type and variant (preemptive/non-preemptive) selection
- Configurable number of processes and per-process parameters
- Dynamic process addition via the '+Add Process' dialog during live execution
- Start Live, Run Instantly, Pause, Stop, and Reset controls
- Pre-loaded test cases for quick demonstrations

5. Scheduling Algorithms — Full Code

5.1 FCFS

```
from core.process import Process
from core.base_scheduler import BaseScheduler

class FCFS(BaseScheduler):
    def __init__(self):
        super().__init__()
        self.ready_queue = []

    def tick(self):
        for p in self.processes:
            if p.arrival_time == self.current_time and p not in self.ready_queue \
               and p != self.current_process and not p.is_completed:
                self.ready_queue.append(p)

        if self.current_process is None and not self.ready_queue:
            self.gantt_chart.append({'pid': 'Idle', 'time': self.current_time})
            self.current_time += 1
            return {'time': self.current_time - 1, 'running': None, 'remaining': {}}

        if self.current_process is None:
            self.current_process = self.ready_queue.pop(0)
            if self.current_process.start_time == -1:
                self.current_process.start_time = self.current_time

        pid = self.current_process.pid
        finished = self.current_process.execute_one_unit()
        if finished:
            self.current_process.completion_time = self.current_time + 1
            self.current_process.calculate_times()
            self.current_process = None

        self.gantt_chart.append({'pid': pid, 'time': self.current_time})
        self.current_time += 1
        return {
            'time': self.current_time - 1,
            'running': self.current_process.pid if self.current_process else None,
            'remaining': {p.pid: p.remaining_time for p in self.processes if not
p.is_completed}
        }

    def reset(self):
        self._base_reset()
        self.ready_queue = []
```

5.2 SJF

```
class SJFScheduler(BaseScheduler):
    def __init__(self, preemptive=False):
        super().__init__()
        self.preemptive = preemptive

    def _ready_queue(self):
        available = [p for p in self.processes
                     if p.has_arrived(self.current_time) and not p.is_completed]
```

```
        key = (lambda p: (p.remaining_time, p.arrival_time, p.pid))
             if self.preemptive
             else (lambda p: (p.burst_time, p.arrival_time, p.pid))
        return sorted(available, key=key)

    def tick(self):
        queue = self._ready_queue()
        if queue:
            top = queue[0]
            if self.preemptive:
                self.current_process = top
            elif self.current_process is None or self.current_process.is_completed:
                self.current_process = top
        else:
            self.current_process = None
        # ... (execute, log Gantt, advance time - identical pattern to FCFS)
```

5.3 Priority

```
class PriorityScheduler(BaseScheduler):
    def __init__(self, preemptive=False):
        super().__init__()
        self.preemptive = preemptive

    def _ready_queue(self):
        return sorted(
            [p for p in self.processes
             if p.has_arrived(self.current_time) and not p.is_completed],
            key=lambda p: (p.priority, p.arrival_time, p.pid)
        )

    def tick(self):
        queue = self._ready_queue()
        if queue:
            top = queue[0]
            if self.preemptive:
                self.current_process = top
            elif self.current_process is None or self.current_process.is_completed:
                self.current_process = top
        else:
            self.current_process = None
        # ... (execute, log Gantt, advance time)
```

5.4 Round Robin

```
from collections import deque

class RoundRobinScheduler(BaseScheduler):
    def __init__(self, quantum=5):
        super().__init__()
        self.quantum = quantum
        self.ready_queue = deque()
        self.time_in_quantum = 0

    def tick(self):
        for p in self.processes:
            if p.arrival_time == self.current_time and p not in self.ready_queue \
               and p != self.current_process and not p.is_completed:
```

```
                self.ready_queue.append(p)

        if self.current_process is None and not self.ready_queue:
            # CPU idle
            self.gantt_chart.append({'pid': 'Idle', 'time': self.current_time})
            self.current_time += 1
            return {'time': self.current_time - 1, 'running': None, 'remaining': {}}

        if self.current_process is None:
            self.current_process = self.ready_queue.popleft()
            self.time_in_quantum = 0
            if self.current_process.start_time == -1:
                self.current_process.start_time = self.current_time

        pid = self.current_process.pid
        finished = self.current_process.execute_one_unit()
        self.time_in_quantum += 1

        if finished:
            self.current_process.completion_time = self.current_time + 1
            self.current_process.calculate_times()
            self.current_process = None
        elif self.time_in_quantum >= self.quantum:
            for p in self.processes:
                if p.arrival_time == self.current_time + 1 \
                   and p not in self.ready_queue and not p.is_completed:
                    self.ready_queue.append(p)
            self.ready_queue.append(self.current_process)
            self.current_process = None

        self.gantt_chart.append({'pid': pid, 'time': self.current_time})
        self.current_time += 1
        return {
            'time': self.current_time - 1,
            'running': self.current_process.pid if self.current_process else None,
            'remaining': {p.pid: p.remaining_time for p in self.processes if not
p.is_completed}
        }

    def reset(self):
        self._base_reset()
        self.ready_queue = deque()
        self.time_in_quantum = 0
```

6. Test Cases & Results

The following standard test case was used to verify correctness across all algorithms. Where Priority Scheduling is tested, lower priority number = higher importance.

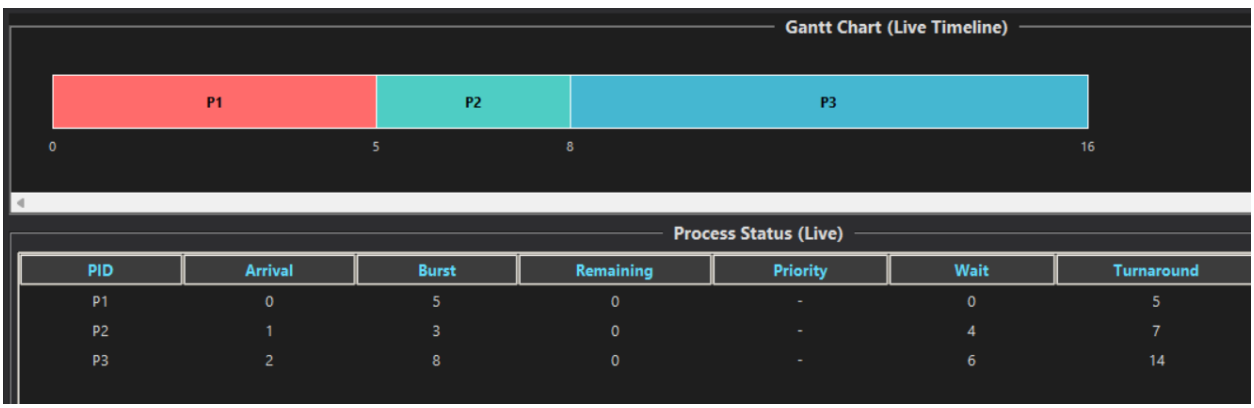
6.1 Test Input

Process	Arrival Time	Burst Time	Priority	Wait (FCFS)	TAT (FCFS)
P1	0	5	3	0	5
P2	1	3	1	4	7
P3	2	8	2	6	14

Table 1 – Test case process set with FCFS results

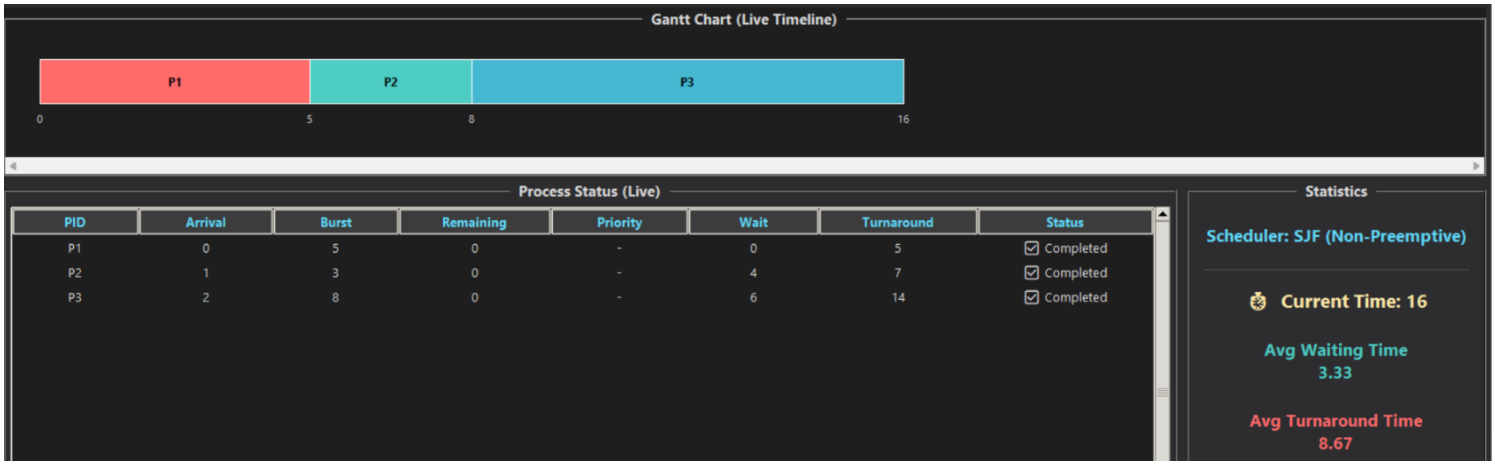
6.2 FCFS Trace

Execution order (non-preemptive, FIFO): P1 → P2 → P3



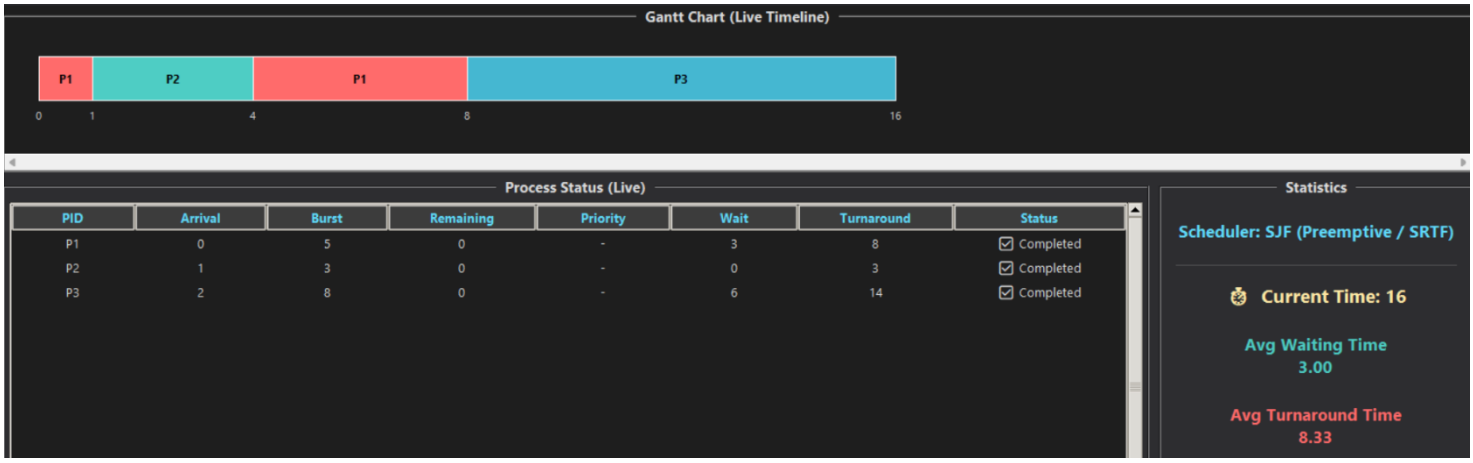
6.3 SJF Non-Preemptive Trace

At t=0 only P1 is available; it runs. At t=5, P2 (burst=3) < P3 (burst=8), so P2 runs next.



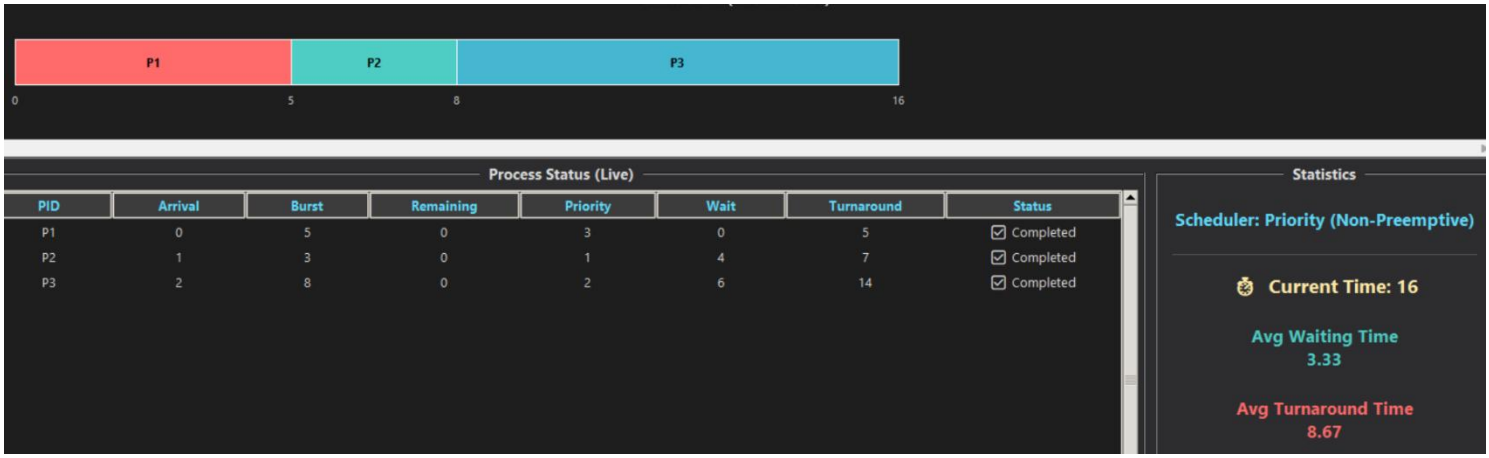
6.4 SJF Preemptive (SRTF) Trace

At t=1 P2 arrives with remaining=3 < P1 remaining=4, so P2 preempts P1.



6.5 Priority Non-Preemptive Trace

P2 has highest priority (1), P3 next (2), P1 lowest (3). At t=0 only P1 is available, so it runs.

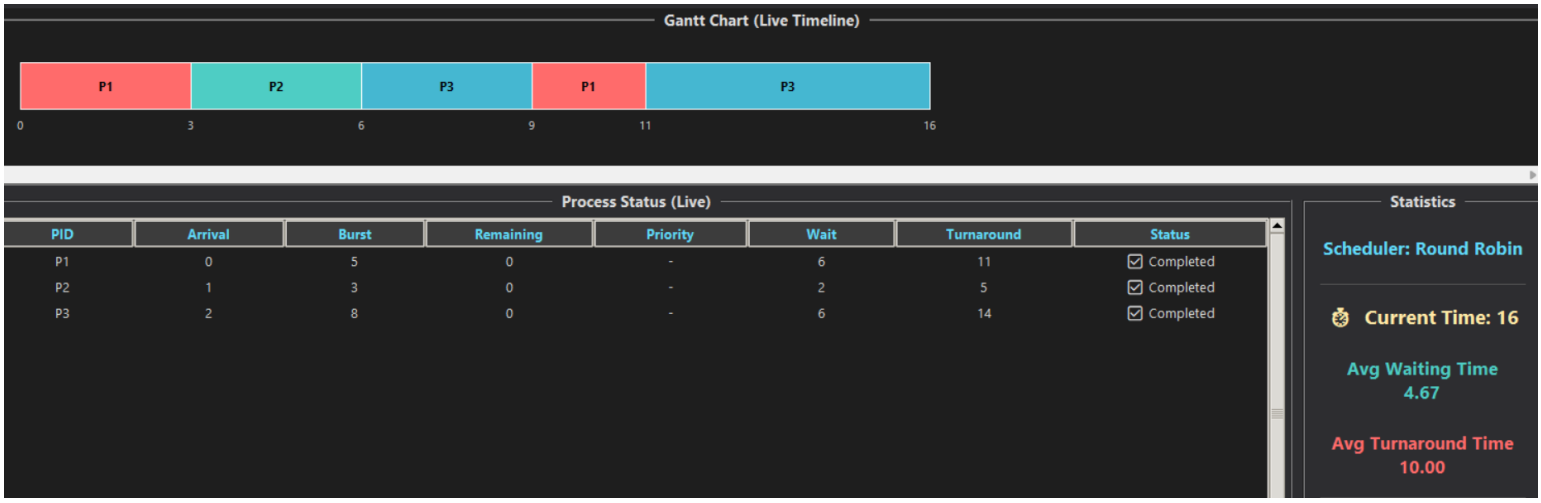


6.6 Priority Preemptive Trace

At t=1, P2 (priority=1) arrives and preempts P1 (priority=3). At t=4 P2 finishes; P3 (priority=2) then preempts P1 (priority=3).

PID	Arrival	Burst	Remaining	Priority	Wait	Turnaround	Status
P1	0	5	0	3	11	16	Completed
P2	1	3	0	1	0	3	Completed
P3	2	8	0	2	2	10	Completed

6.7 Round Robin (Quantum = 3) Trace



PID	Arrival	Burst	Remaining	Priority	Wait	Turnaround	Status
P1	0	5	0	-	6	11	Completed
P2	1	3	0	-	2	5	Completed
P3	2	8	0	-	6	14	Completed

7. Algorithm Comparison

Algorithm	Avg Wait	Avg TAT	Starvation Risk	Fair?
FCFS	3.33	8.67	None	No
SJF (Non-Preemptive)	3.33	8.67	Yes (long jobs)	No
SJF / SRTF (Preemptive)	3.00	8.33	Yes (long jobs)	No
Priority (Non-Preemptive)	3.33	8.67	Yes (low priority)	No
Priority (Preemptive)	4.33	9.67	Yes (low priority)	No
Round Robin (q=3)	4.67	10.00	None	Yes

Table 2 – Algorithm comparison on the standard test case (Priority: lower number = higher importance)

Key observations:

- SRTF achieves the best average waiting time (3.00) as expected from theory
- Round Robin with q=3 has the highest wait time but guarantees no starvation and provides the best response time for short interactive tasks
- Priority Preemptive performs worst in this scenario because P1 (lowest priority) is repeatedly delayed by both P2 and P3
- For this particular input, FCFS, SJF NP, and Priority NP produce identical results because arrival ordering and shortest-job ordering coincide

8. GUI Design

The GUI is built using Python's Tkinter library and is structured into five visual panels that update in real time as the simulation progresses.

The GUI is built using Tkinter and supports interactive scheduling.

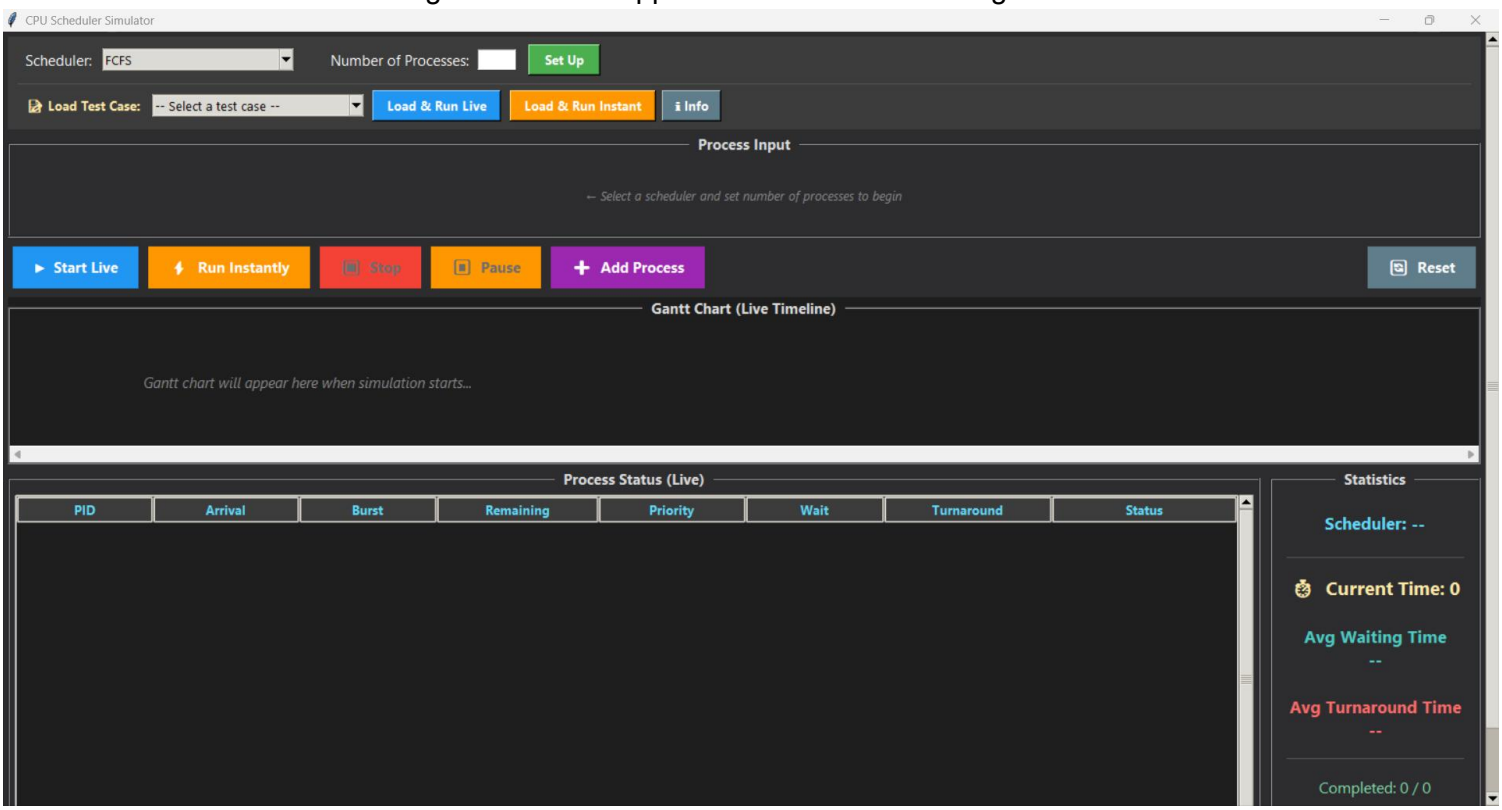


Figure 1-Main interface before simulation starts

Figure 2-Process input panel with user-defined processes

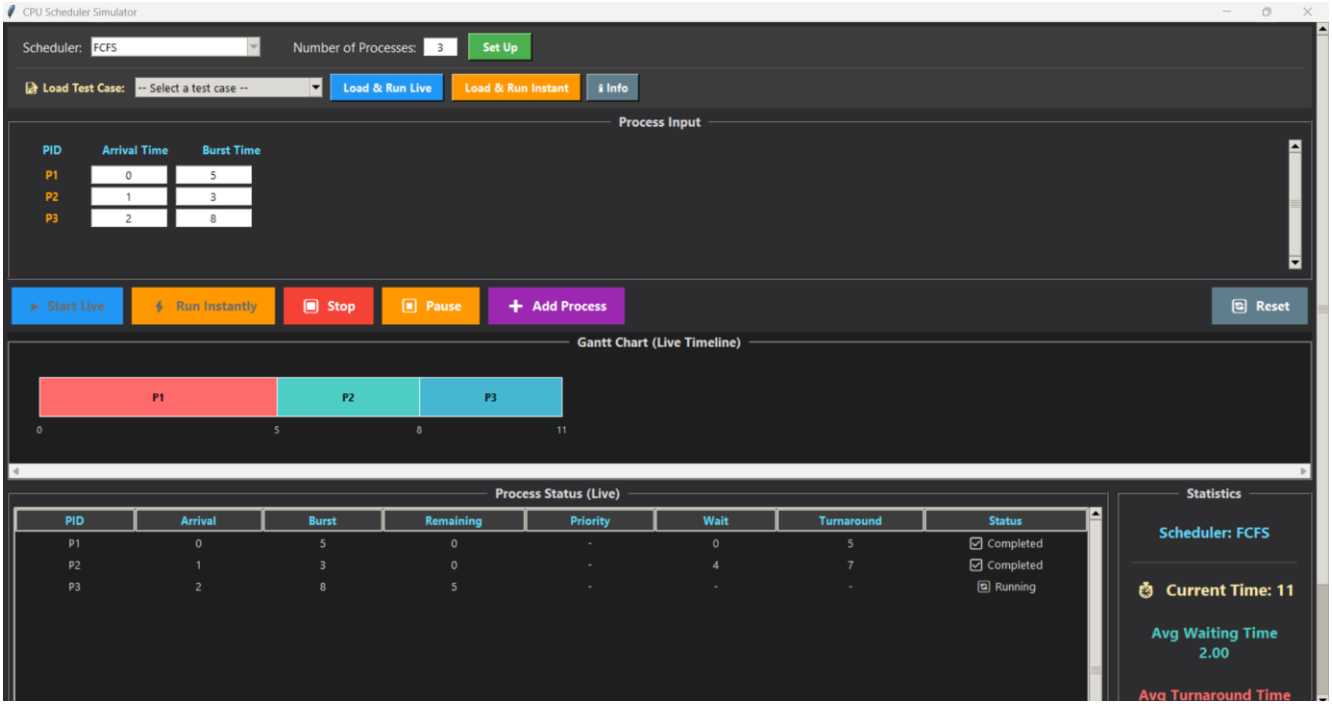


Figure 3-Live simulation showing real-time scheduling and Gantt chart

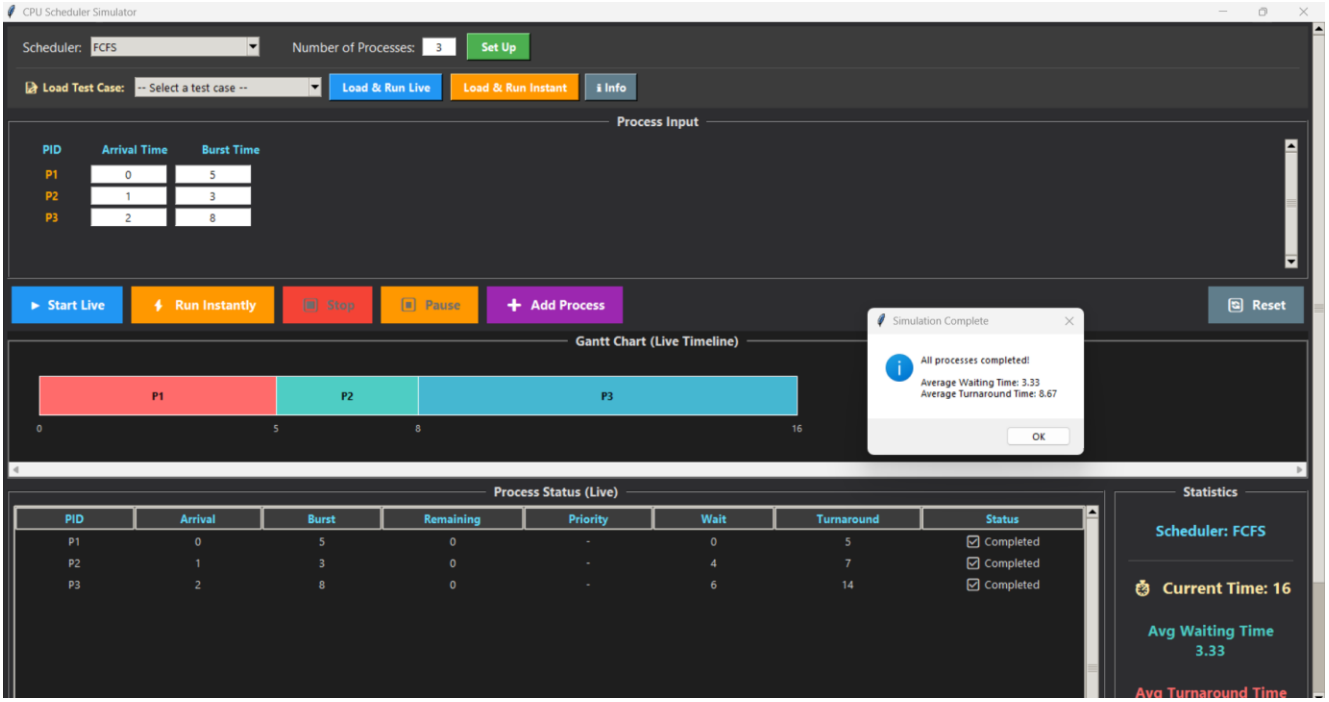


Figure 4-Final Gantt chart after execution

Figure 5-Statistics panel showing average waiting and turnaround times

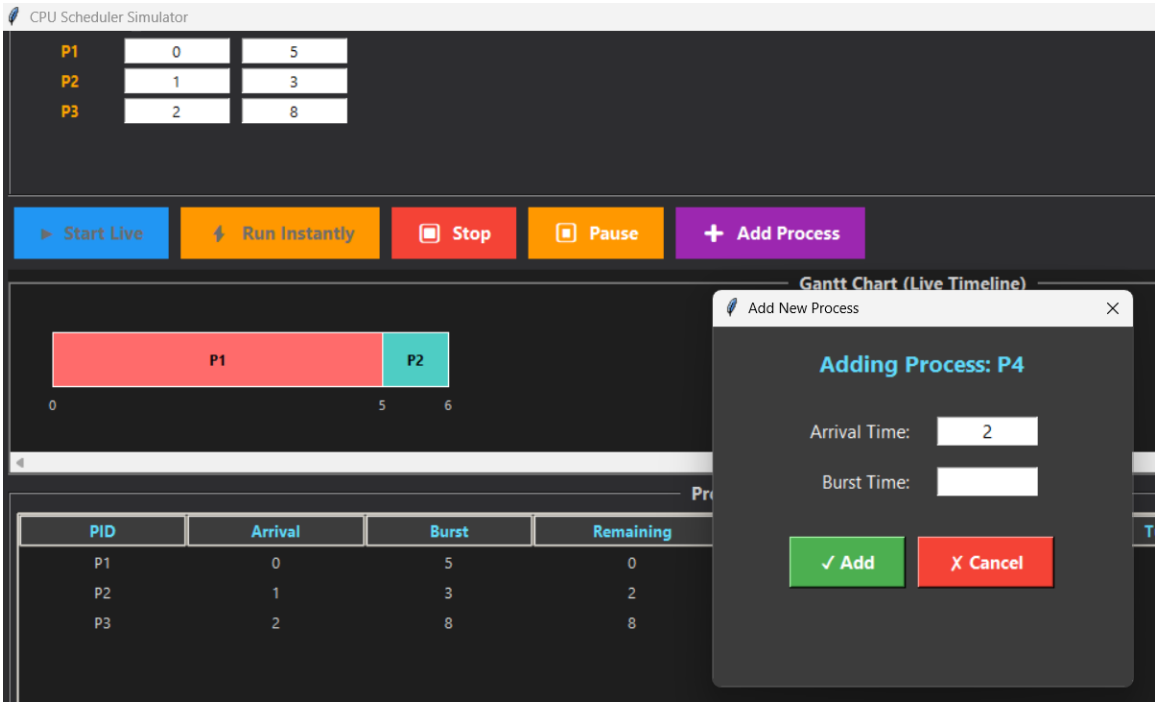


Figure 6-Dynamic process addition during live execution

9. How to Run

9.1 From Source

- Ensure Python 3.9 or later is installed
- No additional packages are required — the project uses only Python standard library modules (tkinter, collections, abc)

```
python main.py
```

9.2 From the Executable

- Download CPU_Scheduler.exe from the Google Drive link
- Double-click the .exe file — no Python installation required
- Windows Defender may show a SmartScreen warning for unsigned executables; click 'More info → Run anyway'

> *The executable was built with PyInstaller and bundles the Python interpreter, Tkinter, and all project modules into a single file.*

10. Conclusion

This project successfully implements a fully functional CPU Scheduling Simulator that supports all required algorithms: FCFS, SJF (preemptive and non-preemptive), Priority (preemptive and non-preemptive), and Round Robin with a configurable quantum.

The modular three-layer architecture cleanly separates scheduling logic, simulation control, and user interface concerns, making the codebase easy to extend. The live GUI provides intuitive visual feedback through a real-time Gantt chart, a live process status table, and a statistics panel.

The comparative analysis on the standard test case confirms the expected theoretical trade-offs: SRTF achieves the lowest average waiting time at the cost of potential starvation; Round Robin eliminates starvation at the cost of higher average wait; and FCFS remains simple but susceptible to the convoy effect. These results validate the correctness of the implementation and provide a practical illustration of the scheduling concepts taught in the Operating Systems cours

GitHub:

amr10w/CPU-Scheduler